

Architectural Health Scoring with Inferred Module Contracts and Explicit Non-Measurement Reporting

Harvik Sanghavi

*Department of Data Science
Mukesh Patel School of
Technology Management and
Engineering, NMIMS
Mumbai, Maharashtra, India
harvik07sanghavi@gmail.com*

Jainam Sethia

*Department of Data Science
Mukesh Patel School of
Technology Management and
Engineering, NMIMS
Mumbai, Maharashtra, India
jainamsethia19@gmail.com*

Vansh Parikh

*Department of Data Science
Mukesh Patel School of
Technology Management and
Engineering, NMIMS
Mumbai, Maharashtra, India
vanshparikh@gmail.com*

Rashmi Patel

*Department of Data Science
Mukesh Patel School of
Technology Management and
Engineering, NMIMS
Mumbai, Maharashtra, India
rashmi.patel@nmims.edu*

Abstract—Architectural erosion is usually detected long after it becomes expensive to reverse, and the tools that could detect it earlier require a hand written description of the intended architecture that most repositories do not have. This paper presents a repository level analysis pipeline that removes that prerequisite. Module boundaries are inferred from version history using community detection over a co change graph, with a directory based fallback when history is too sparse, and the inferred boundaries are then checked by four analysis layers covering import rules, structural coupling, semantic drift and cross module duplication. Semantic layers use sentence embeddings and an approximate nearest neighbour index rather than token comparison. The central design decision is that a layer which cannot measure anything reports the reason instead of a score, and the composite is averaged only over layers that actually ran. The system was applied to ten public Python repositories totalling 102331 lines, twice each, and every repository was analysed end to end without manual configuration. The measurements show that on first contact only two of four layers produce a value, that a repeat scan activates a third, and that the resulting composite rose on five of ten repositories purely because the set of contributing layers changed rather than because the code improved. The evaluation therefore supports feasibility of unattended analysis and identifies a comparability constraint that any active layer reweighting scheme must address. Claims about explanation quality are not made, because that component was not evaluated.

Keywords—*software architecture; architectural erosion; static analysis; code embeddings; similarity search; repository mining; software metrics*

I. INTRODUCTION

Software architecture degrades gradually. Each individual change is defensible, and the loss of structure only becomes visible once a module has acquired more dependencies than any one developer can reason about. Perry and Wolf identified this as architectural erosion and drift [1], Lehman described the underlying pressure as a law of continuing change [2], and van Gorp and Bosch documented design erosion in practice [3]. The cost is well understood in the technical debt literature [4], [5].

The tooling response has largely been conformance checking: a developer states the intended architecture, and a checker reports deviations. This works, and it is the basis of established dependency rule enforcement. It also has a practical prerequisite that is rarely met. The specification must exist before the first check can run, and writing one for an existing repository requires precisely the architectural

understanding that the repository has already lost. Projects that would benefit most from conformance checking are therefore the least likely to adopt it. Surveys of static analysis adoption report related friction: developers abandon tools whose output they cannot act on or whose configuration cost is paid before any value is returned [6], [7].

A second difficulty concerns what an analysis reports when it cannot run. A layer that finds no rules to enforce, or no historical baseline to compare against, has not observed a healthy repository; it has observed nothing. If that absence is encoded as a zero and averaged into an aggregate score, the aggregate improves as coverage falls. This is a measurement validity problem rather than an implementation defect, and it is not specific to any one tool.

This paper presents a pipeline addressing both points. Module boundaries are inferred automatically, so no specification is required in advance, and every layer distinguishes a measured value from an absence of measurement, so the aggregate is computed only over layers that produced evidence. The system was applied to ten public Python repositories, twice each, and the measurements are reported without a claim that high scores indicate good architecture.

The contributions are: a method for unattended module contract inference combining version history with a structural fallback; an aggregation rule that makes non measurement explicit rather than silently favourable; an implemented system realising both; and an empirical characterisation of layer activation that identifies a comparability constraint the aggregation rule introduces.

II. RELATED WORK

A. Software architecture recovery

Recovering a module view from source is a mature field. Ducasse and Pollet survey the space and organise it by the goal of the reconstruction and the inputs it consumes [8]; Garcia et al. compare recovery techniques empirically and report that accuracy varies substantially with the system under study [9]. Clustering over structural dependencies is the dominant input. The pipeline described here differs in the input rather than the algorithm: it clusters a co change graph derived from version history, following the observation of Gall et al. that files changing together are logically coupled even when no static dependency connects them [10], and the later confirmation by Zimmermann et al. that such histories predict change propagation [11]. Recovery here is a means to an end. The

inferred grouping exists so that later layers have a unit to measure, not as a claim about the true architecture.

B. Coupling metrics and erosion detection

Chidamber and Kemerer established coupling and cohesion as measurable object oriented properties [12], and Martin's dependency metrics introduced the instability and abstractness formulation that underlies most modern dependency rule checking [13]. These give absolute measures of structure at a point in time. The coupling layer described here measures the same property but against a declared budget rather than as a bare figure, so what it reports is the distance from a stated policy. Two policies are available, one derived from the repository's own state and one fixed; Section IV explains why unattended analysis uses the fixed one, and Section VIII examines what that costs in discrimination.

C. Clone detection and code embeddings

Token and tree based clone detection is well established, with CCFinder as a representative large scale implementation [14] and comparative surveys documenting the trade offs between clone types and detection techniques [15]. Studies of clone prevalence at ecosystem scale show duplication is pervasive rather than exceptional [16]. Purely lexical techniques, however, miss reimplementations that share intent but not tokens. Learned representations address this: deep learning has been applied directly to clone detection [17], and pretrained code models such as CodeBERT [18] and GraphCodeBERT [19] provide general purpose code embeddings. The system described here uses a general sentence embedding model, MiniLM [20] in the Sentence BERT configuration [21], over function level text, indexed with an approximate nearest neighbour structure [22]. This is a weaker representation than a code specific model, and Section IX treats that as a limitation rather than a design advantage.

D. Developer facing analysis tooling

Sadowski et al. describe the organisational requirements for static analysis to survive contact with developers, emphasising low friction and actionable output [6]. Johnson et al. found that unclear or unactionable output is a primary reason tools are abandoned [7]. Both motivate the reporting decisions here: a layer that did not run states why in the same place it would have stated a score.

E. Gap addressed

Architecture recovery produces module views but stops short of continuous enforcement; conformance checking enforces but requires a specification; clone detection addresses similarity in isolation from architectural structure. The combination pursued here is one unattended pipeline that infers the unit of analysis, measures it structurally and semantically, and is explicit about which measurements were possible. How often each layer can measure anything appears not to be routinely reported, and that is what Section VII answers here.

III. RESEARCH QUESTIONS

The evaluation is limited to questions the available evidence can answer. Explanation quality and the accuracy of inferred boundaries against a ground truth are outside it, for the reasons given in Section IX.

RQ1. Can module contracts be inferred well enough for a four layer analysis to complete end to end on unseen repositories without manual configuration?

RQ2. Which layers actually produce a measurement on first contact with an unseen repository, and does that change on a repeat scan?

RQ3. What effect does averaging over only the active layers have on the comparability of the composite score across scans?

RQ4. What is the wall clock cost of the pipeline, and how much of it is avoided by caching on a repeat scan?

IV. PROPOSED METHOD

Fig. 1 shows the pipeline. A repository is cloned, parsed into an import graph, partitioned into modules, planned incrementally, analysed by four layers, scored, and persisted.

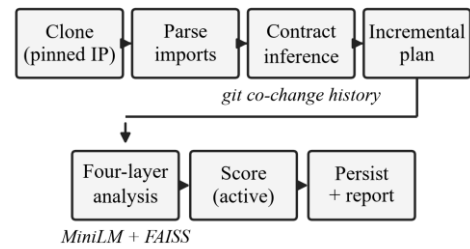


Fig. 1. Analysis pipeline. Italic labels mark the two inference inputs: version history for module boundaries and embeddings for the semantic layers. The score is the mean over active layers only.

A. Module contract inference

The unit of measurement is a module: a named set of file paths. When no specification is present one is generated. Non merge commits are read from version history and each commit contributes an edge between every pair of Python files it touches. Two bounds keep this tractable and meaningful. Only the most recent 500 commits are converted to edges once a repository exceeds 1000 commits, and any commit touching more than 100 files is counted but contributes no edges, since bulk reformatting and vendored tree imports would otherwise dominate the graph with edges carrying no coupling signal. Louvain modularity optimisation [23] partitions the resulting graph, and communities are consolidated when they resolve to the same directory path. Where history is too sparse, boundaries fall back to directory structure. Candidate modules that resolve to no existing path are dropped; if none survives, the analysis stops and reports that it could not identify a module, rather than scoring an empty partition.

B. Layer 1: import boundary violations

Each file is assigned to a module and its import statements are checked against the forbidden import rules the contract declares. The layer score is the proportion of examined imports that breached a rule:

$$L1 = violations / \max(imports_examined, 1)$$

A contract that declares no import rules puts no file in front of a rule. The layer then reports that it examined nothing, rather than the zero that the expression above would otherwise yield.

C. Layer 2: structural coupling

Fan out is the number of distinct non standard library, non relative modules a module imports. Each module carries a coupling budget, and the per module delta and layer score are

$$\text{delta}(m) = \min(1, (\text{fan_out}(m) - \text{budget}(m)) / \max(\text{budget}(m), 1))$$

$$L2 = \max \text{ over modules } m \text{ of } \text{delta}(m)$$

Where the budget comes from is the more consequential choice, and the generator offers two policies. A team enforcing a contract against its own future changes can derive the budget from the module's fan out at generation time, at 1.5 times the observed value. For one off analysis of a repository nobody has agreed a contract for, that policy is rejected in favour of a fixed threshold, on the explicit grounds that grading a repository against thresholds derived from its own current state is tautological and can only pass. The hosted path used throughout this evaluation therefore applies a fixed profile that sets a budget of ten for every module, so Layer 2 measures observed coupling against a stated policy rather than against the repository itself.

D. Layer 3: semantic drift

Function level text is embedded with all MiniLM L6 v2, a 384 dimensional sentence embedding model [20], [21], used as a pretrained encoder; no training or fine tuning is performed. Embeddings are computed in batches of 32 and cached by content hash. A module centroid is the mean of its function embeddings normalised to unit length, and drift is the cosine distance between the stored centroid and the current one:

$$L3 = 1 - \text{cosine_similarity}(\text{centroid_previous}, \text{centroid_current})$$

A module exceeding its drift threshold raises a violation, and the layer score is the maximum drift across modules. The threshold is set by the same profile that sets the coupling budget, as one minus the profile's minimum cohesion, which is 0.35 for the profile used here. On a repository with no stored centroid there is nothing to compare against; the layer records the new centroid, reports that no baseline existed and contributes no score.

E. Layer 4: cross module duplication

Function embeddings are unit normalised and indexed in a flat inner product index over exact L2 distance [22], from which cosine similarity is recovered as

$$\text{cosine_similarity} = 1 - (\text{L2_distance}^2) / 2$$

Each function queries its ten nearest neighbours. Candidates below 0.70 similarity are discarded, matches are retained only across module boundaries, and a match is scored by linear interpolation above a similarity floor:

$$\text{dup}(s) = \text{clamp}((s - 0.85) / 0.15, 0, 1)$$

A module aggregates the mean of its match scores and the layer takes the maximum across modules. The floor at 0.85 means near identical implementations score close to one while merely related functions score zero, which keeps the layer from reporting ordinary domain similarity as duplication.

F. Composite score and non measurement

Let A be the set of layers that produced a measurement. The composite is their unweighted mean and the reported health score is its complement on a percentage scale:

$$\text{composite} = (1 / |A|) * \text{sum of layer scores over } A$$
$$\text{health} = (1 - \text{composite}) * 100$$

When A is empty the composite is set to 1.0 rather than 0.0. Averaging over no layers is arithmetically zero, which would report a repository on which nothing could be checked as perfectly healthy; of the two available answers, the conservative one is the only defensible choice. Bands are contract relative: a composite at or above the contract's fail threshold is critical, at or above the warn threshold is warning, at or above half the warn threshold is watch, and below that is healthy. A failed critical fitness gate caps the reported band and letter grade without altering the composite, so a breach cannot be presented as healthy while the numeric measurement remains what was measured.

G. Incremental analysis

Files are hashed with SHA-256 and compared against the previous run to partition them into changed and unchanged sets, from which the set of dirty modules follows. The contract is fingerprinted so that a change to the specification invalidates reuse. Layer 4 is recomputed repository wide whenever any module is dirty, because duplication is a cross module property and a partial index would silently miss matches whose counterpart lies in an unchanged module.

V. SYSTEM IMPLEMENTATION

The system is implemented in Python in 16534 non blank lines across 116 modules, with a test suite of 20529 lines across 146 modules, a ratio of 1.24 test lines per source line. It is deployed as two container images from one definition: a web image serving an HTTP API and dashboard, and a worker image carrying the machine learning dependencies and the embedding model. Analysis runs only in the worker, reached through a durable queue, so that untrusted repository parsing never executes in the process holding user sessions. Results are persisted in PostgreSQL; the queue, sessions and rate limit counters are held in Redis.

Two pieces of per repository state live in PostgreSQL rather than in the clone: content hashes, which drive incremental reuse, and module centroids, which give Layer 3 its baseline. Both were once written into a cache inside the analysed repository, which the hosted path deletes when the job ends, so neither survived to be read again. Keying them by repository is what lets an incremental scan or a drift comparison happen outside a retained working copy.

Several controls are relevant to a system that clones arbitrary repositories. Submitted URLs are resolved and the resolved address is pinned for the outbound request, so that the address validated against the private address blocklist is the address actually contacted, closing a time of check to time of use gap without disabling certificate verification. Clones are depth limited and bounded by a workspace budget. Dependency auditing declines to resolve a project that ships only a build specification, because resolution would execute the repository's own build backend. Responses carry a per request content security policy nonce. Each of these has a corresponding regression test.

Continuous integration runs ten jobs, expanding to eleven runs across a two version Python matrix, covering linting and type checking, dependency vulnerability auditing and static security analysis, unit and integration tests against real PostgreSQL and Redis services, a dedicated job that installs the machine learning extras and asserts that no test skipped for

want of them, migration round trips, container build verification, and browser based accessibility, behavioural and visual regression suites. With the machine learning extras present 1372 tests pass and one skips by design; with them absent branch coverage is 84.59 percent. The system additionally analyses its own repository as a release gate, reporting 98.1 of 100 over 274 files with all eight declared fitness gates passing; this is a self consistency check and not independent evidence.

A language model component provides natural language advisory responses and remediation drafts through an external API, and degrades to a disabled state when no key is configured. It was not evaluated in this work and no claim is made about the quality or usefulness of its output.

VI. EXPERIMENTAL SETUP

Ten public Python repositories were selected to span three orders of magnitude of commit history and roughly two of size, while remaining within one language and one packaging ecosystem so that parser coverage is not a confounding variable. They are widely used libraries and one tutorial application, chosen before any result was observed and analysed without exception or retry. Table I reports their characteristics. The corpus totals 499 Python files and 102331 non blank lines.

TABLE I. EVALUATION CORPUS CHARACTERISTICS

Repository	Commits	Files	Lines
itsdangerous	677	15	1,359
cachecontrol	654	31	2,829
click	3,362	90	23,803
flask	5,557	83	14,085
requests	6,494	37	9,841
starlette	1,737	84	17,275
microblog	25	34	1,507
records	378	9	764
httpx	1,523	60	14,376
attrs	1,841	56	16,492

Each repository was cloned and analysed twice in the same working copy through the same pipeline entry point the production worker uses. The first scan is the first contact condition: no contract, no cached embeddings, no stored centroid. The second scan is the repeat condition: the contract generated by the first scan is present, as are its cached embeddings and centroids. Wall clock duration was measured around the pipeline call. Measurements were taken on one workstation running Python 3.11 with the machine learning extras installed; the embedding model is loaded once per process, so the first repository analysed absorbs that one off cost and is identified as such in Fig. 3.

Reported quantities are those the system itself produces: per layer score, whether each layer was skipped and why, violation counts, composite, health score and duration. No ground truth for architectural quality was available, so no precision or recall is reported; Section IX states why.

Data availability. Both scan drivers, the raw results they emitted, the figure scripts and the script that typeset this paper from those results are in the repository, under *paper/*: github.com/jainamsethia/ArchGuard

VII. RESULTS

A. Completion without configuration

All ten repositories were analysed end to end on both scans with no manual configuration and no hand authored contract. Contract inference produced a usable partition in every case, and no run terminated in the degenerate state where no candidate module resolves to an existing path. This answers RQ1 affirmatively for this corpus, and establishes only that the pipeline completes, not that the inferred boundaries are correct.

B. Which layers measured anything

The activation result is uniform and is the most consequential finding. On the first scan the active set was layers 2 and 4 for all ten repositories. Layer 1 reported that the contract declared no import rules, and layer 3 reported that no prior baseline existed. On the repeat scan the active set was layers 2, 3 and 4 for all ten: the centroid stored by the first scan gave layer 3 something to compare against. Layer 1 was inactive throughout. Table II gives the first scan outcome per repository.

TABLE II. FIRST SCAN LAYER OUTCOMES

Repository	L2	L4	Comp.	Health	Gr.
itsdangerous	0.000	0.000	0.000	100.0	A
cachecontrol	0.000	0.000	0.000	100.0	A
click	0.000	0.000	0.000	100.0	A
flask	0.100	0.000	0.050	95.0	A
requests	0.000	0.000	0.000	100.0	A
starlette	0.200	0.000	0.100	90.0	A
microblog	1.000	0.000	0.500	50.0	F
records	0.000	0.000	0.000	100.0	A
httpx	0.500	0.000	0.250	75.0	C
attrs	0.000	0.076	0.038	96.2	A

Layer 2 produced at least one violation on four of ten repositories and layer 4 on one. The single layer 4 result was two cross module matches in one repository with an aggregate of 0.076, well below the 0.85 similarity floor at which the score approaches one. Of the four layers, therefore, one never measured, one measured only after a baseline existed, one fired on four repositories and one fired on one.

C. Effect of reweighting on comparability

Fig. 2 compares health scores across the two conditions. Five of ten repositories scored higher on the repeat scan. No source file changed between scans. The increase follows arithmetically from the aggregation rule: the composite is the mean over the active set, and adding a third active layer whose score is near zero lowers the mean. For the lowest scoring repository the composite moved from 0.500, the mean of 1.000 and 0.000 over two layers, to 0.333, the mean of the same two values and a third near zero drift score, raising reported health from 50.0 to 66.7 with no change to the code.

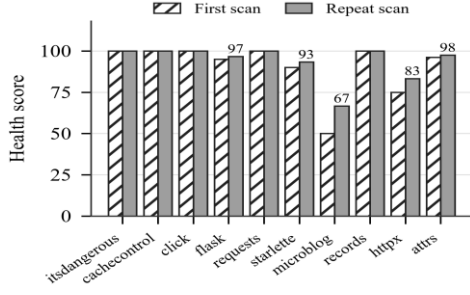


Fig. 2. Health score per repository on first and repeat scan. Repeat scan values are labelled where they differ. No source file changed between the two scans.

Across the corpus, mean health rose from 90.6 to 93.8 and the standard deviation fell from 15.4 to 10.3. Both movements are artefacts of the change in the active set, not evidence of improvement. This answers RQ3: composite scores computed over different active sets are not directly comparable, and the aggregation rule that prevents unmeasured layers from inflating a score introduces a distinct comparability hazard of its own.

D. Cost

Fig. 3 reports wall clock analysis time. Excluding the first repository, which absorbs the one off model load, the first scan averaged 13.1 seconds and the repeat scan 8.3 seconds, a mean per repository reduction of 45 percent with a range of 21 to 78 percent. The largest repository in the corpus, at 23803 lines, completed its first scan in 31.3 seconds. Absolute times are single machine measurements and indicate an order of magnitude, not a benchmark.

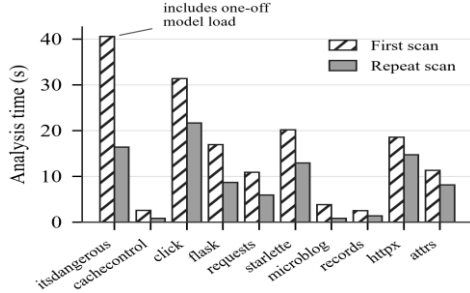


Fig. 3. Wall clock analysis time per repository. The first bar includes the one off embedding model load paid once per process.

TABLE III. FIRST VERSUS REPEAT SCAN

Repository	Active layers	Health	Time (s)
itsdangerous	2,4 → 2,3,4	100.0 to 100.0	40.6 to 16.4
cachecontrol	2,4 → 2,3,4	100.0 to 100.0	2.6 to 0.8
click	2,4 → 2,3,4	100.0 to 100.0	31.3 to 21.7
flask	2,4 → 2,3,4	95.0 to 96.7	16.9 to 8.7
requests	2,4 → 2,3,4	100.0 to 100.0	10.9 to 5.9
starlette	2,4 → 2,3,4	90.0 to 93.3	20.2 to 12.9
microblog	2,4 → 2,3,4	50.0 to 66.7	3.8 to 0.8
records	2,4 → 2,3,4	100.0 to 100.0	2.5 to 1.4
httpx	2,4 → 2,3,4	75.0 to 83.3	18.6 to 14.7
attrs	2,4 → 2,3,4	96.2 to 97.5	11.3 to 8.1

VIII. DISCUSSION

A. Two of four layers dominate the result

The headline design is a four layer analysis; the measured behaviour on first contact is a two layer analysis, and effectively a one layer analysis given that layer 4 fired on a single repository. The causes are structural rather than

incidental. Layer 1 is inactive because contract inference declines to emit import rules at all, and declines for a stated reason: a dependency cycle proves that at least one edge in it is wrong without identifying which, so naming a particular edge as forbidden would be a guess presented as a rule. The generator emits a cycle detecting fitness function instead, which reports the path it found rather than inventing a policy. Layer 1 therefore has nothing to enforce by construction rather than by omission. Layer 3 is inactive on first contact because drift is defined against a previous observation, and a first observation has no predecessor. Neither is a wrong answer, and the system reports both, but together they mean a single unattended scan of an unseen repository exercises far less of the pipeline than its description implies. Reporting only a composite would have concealed this entirely.

B. What a high score does and does not mean

The scores cluster high, and the obvious explanation is wrong. It is not that the contract ratifies the repository: the generator explicitly rejects self derived thresholds for one off analysis, and the profile used here fixes every module's budget at ten regardless of what the repository does. The four violations are genuine exceedances of that stated policy, at fan outs of 11, 12, 15 and 21 against a budget of 10.

What the clustering reflects is that a budget of ten is permissive for this kind of repository: six of ten mature libraries keep every module below it, which is a plausible property of well factored code rather than an artefact. The limitation is one of discrimination rather than validity. At this threshold the layer separates the four most coupled repositories from the rest and says nothing about the ordering within either group; a stricter profile would separate more, at the cost of failing healthy libraries. That trade is a policy choice the profile mechanism exposes, not a question this evaluation settles.

C. Semantic layers and their yield

The semantic machinery accounted for a large share of dependency weight and runtime and produced findings on one of ten repositories. Two readings are available and this evaluation cannot separate them. The libraries in the corpus are mature and well factored, so genuine cross module duplication may simply be rare; alternatively the 0.85 similarity floor, chosen to suppress ordinary domain similarity, may be too conservative. Distinguishing these requires a labelled duplication benchmark, which is future work. A comparable ambiguity applies to layer 3, whose activation on the repeat scan was demonstrated but whose drift values were near zero because the two scans observed identical code; the layer has been shown to activate, not to detect real drift.

D. Threats to validity

Construct. Health score is an aggregate of four heuristics and is not validated against any external judgement of architectural quality; it measures conformance to an inferred contract. *Internal.* Both scans ran in the same working copy, so the repeat condition confounds three caches that were all populated by the first scan. The activation change is attributable to the stored centroid because the layer states its own reason, but runtime reduction cannot be attributed to a single cache. *External.* Ten Python repositories, nine of them libraries, do not generalise to other languages, to applications, or to repositories under active architectural change. *Conclusion.* The corpus is too small for inferential statistics,

and none is reported; the activation results are uniform across all ten repositories, which is the strongest claim the design supports.

IX. LIMITATIONS

The parser handles Python only, and non Python files in a repository are excluded, so a polyglot repository is analysed on a partial view. Repositories must be public and reachable over HTTPS.

No ground truth was available for either inferred module boundaries or architectural quality, so the evaluation reports what the system measured and does not report accuracy. Establishing accuracy would require expert labelled module decompositions, and validating the health score would require an independent quality judgement per repository.

Layer 3's inactivity on a first scan is inherent, and the baseline is stored per repository rather than in the analysed clone, so the repeat condition reported here is the one a hosted deployment reaches on a second submission. What that condition does not establish is detection: both scans observed identical code, so drift was near zero by construction. The layer was shown to activate and to compare, not to discriminate.

The embedding model is a general sentence encoder rather than a code specific one [18], [19], and function text is embedded without structural context. Duplication thresholds were fixed by design rather than calibrated against a benchmark such as a curated clone corpus. The language model component is unevaluated. The self analysis figure is not independent evidence, since the system, the contract and the thresholds share an author.

X. CONCLUSION AND FUTURE WORK

Architectural conformance checking is most valuable where an architectural specification is least likely to exist. This paper described a pipeline that removes that prerequisite by inferring module boundaries from version history with a structural fallback, checks them with four structural and semantic layers, and treats the inability to measure as a reportable outcome rather than a zero. Applied to ten public Python repositories twice each, the system completed every analysis without manual configuration.

The measurements qualify that result in a way the design alone would not reveal. On first contact only two of four layers produced a value; a repeat scan activated a third; and the composite rose on five of ten repositories solely because the set of contributing layers changed. Averaging over active layers prevents unmeasured layers from inflating a score, but makes scores computed over different active sets incomparable. A practical response is to report the active set alongside every score and to refuse comparison across differing sets, which the underlying data already supports.

Future work follows directly from the limitations. Layer 1 is the harder case and may not be a case at all: no import rule appears derivable from an unseen repository without either guessing which edge of a cycle is wrong or allowlisting the edges that already exist, and the second only ever passes on the scan that created it. Whether a fourth layer that cannot act unattended belongs in an unattended pipeline is a design question this evaluation raises rather than answers. Layer 3 is tractable: with baselines now persisted, drift detection can be evaluated across two commits with a known refactoring

between them. Calibrating the duplication threshold against a labelled clone benchmark would separate a conservative threshold from a genuinely clean corpus. Extending the parser beyond Python would test whether the co change inference generalises where import structure differs. Finally, the explanation component requires a study with developers before any claim about its usefulness can be made.

REFERENCES

- [1] D. E. Perry and A. L. Wolf, "Foundations for the study of software architecture," ACM SIGSOFT Software Engineering Notes, vol. 17, no. 4, pp. 40-52, 1992.
- [2] M. M. Lehman, "Programs, life cycles, and laws of software evolution," Proceedings of the IEEE, vol. 68, no. 9, pp. 1060-1076, 1980.
- [3] J. van Gorp and J. Bosch, "Design erosion: problems and causes," Journal of Systems and Software, vol. 61, no. 2, pp. 105-119, 2002.
- [4] W. Cunningham, "The WyCash portfolio management system," in Proc. OOPSLA, 1992, pp. 29-30.
- [5] P. Kruchten, R. L. Nord, and I. Ozkaya, "Technical debt: from metaphor to theory and practice," IEEE Software, vol. 29, no. 6, pp. 18-21, 2012.
- [6] C. Sadowski, J. van Gogh, C. Jaspan, E. Soederberg, and C. Winter, "Tricorder: building a program analysis ecosystem," in Proc. ICSE, 2015, pp. 598-608.
- [7] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?" in Proc. ICSE, 2013, pp. 672-681.
- [8] S. Ducasse and D. Pollet, "Software architecture reconstruction: a process-oriented taxonomy," IEEE Trans. Software Engineering, vol. 35, no. 4, pp. 573-591, 2009.
- [9] J. Garcia, I. Ivkovic, and N. Medvidovic, "A comparative analysis of software architecture recovery techniques," in Proc. ASE, 2013, pp. 486-496.
- [10] H. Gall, K. Hajek, and M. Jazayeri, "Detection of logical coupling based on product release history," in Proc. ICSM, 1998, pp. 190-198.
- [11] T. Zimmermann, P. Weissgerber, S. Diehl, and A. Zeller, "Mining version histories to guide software changes," IEEE Trans. Software Engineering, vol. 31, no. 6, pp. 429-445, 2005.
- [12] S. R. Chidamber and C. F. Kemerer, "A metrics suite for object oriented design," IEEE Trans. Software Engineering, vol. 20, no. 6, pp. 476-493, 1994.
- [13] R. C. Martin, Agile Software Development: Principles, Patterns, and Practices. Upper Saddle River, NJ, USA: Prentice Hall, 2003.
- [14] T. Kamiya, S. Kusumoto, and K. Inoue, "CCFinder: a multilingual token-based code clone detection system for large scale source code," IEEE Trans. Software Engineering, vol. 28, no. 7, pp. 654-670, 2002.
- [15] C. K. Roy, J. R. Cordy, and R. Koschke, "Comparison and evaluation of code clone detection techniques and tools: a qualitative approach," Science of Computer Programming, vol. 74, no. 7, pp. 470-495, 2009.
- [16] C. V. Lopes et al., "DéjàVu: a map of code duplicates on GitHub," Proc. ACM on Programming Languages, vol. 1, no. OOPSLA, pp. 1-28, 2017.
- [17] M. White, M. Tufano, C. Vendome, and D. Poshyanyk, "Deep learning code fragments for code clone detection," in Proc. ASE, 2016, pp. 87-98.
- [18] Z. Feng et al., "CodeBERT: a pre-trained model for programming and natural languages," in Findings of EMNLP, 2020, pp. 1536-1547.
- [19] D. Guo et al., "GraphCodeBERT: pre-training code representations with data flow," in Proc. ICLR, 2021.
- [20] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: deep self-attention distillation for task-agnostic compression of pre-trained transformers," in Proc. NeurIPS, 2020, pp. 5776-5788.
- [21] N. Reimers and I. Gurevych, "Sentence-BERT: sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, 2019, pp. 3982-3992.
- [22] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535-547, 2021.
- [23] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," Journal of Statistical Mechanics: Theory and Experiment, vol. 2008, no. 10, P10008, 2008.